

Student Meal Planner

ERD & Database Design

A Design Document for the Grubs4Scrubs Project

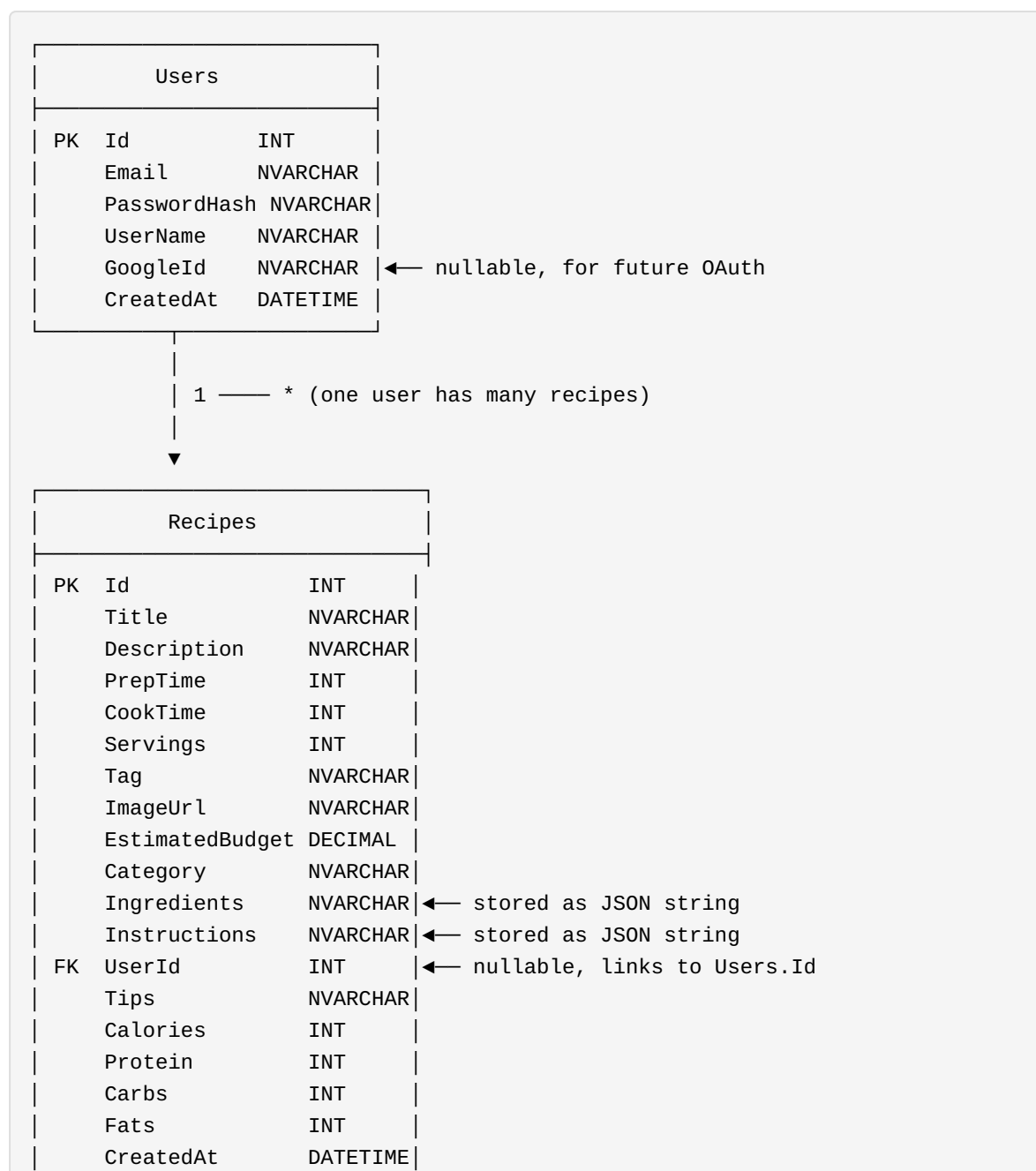
Date: April 2026

Author: Yao Cheng Zhou

Why this document exists

Before writing any SQL or repository code, I needed to think about what data the app actually stores and how the tables relate to each other. This doc covers the current database schema, the relationships between tables, and a few design decisions that aren't obvious from the SQL alone.

Entity Relationship Diagram



ShoppingItems		
PK	Id	INT
	Name	NVARCHAR
	Quantity	NVARCHAR
	Price	DECIMAL
	IsChecked	BIT
FK	UserId	INT ← nullable, links to Users.Id

Tables in detail

Users

Stores registered accounts. The `PasswordHash` column holds a BCrypt hash, never a plain-text password. `GoogleId` is there for a future Google OAuth flow but stays null for email/password accounts. `CreatedAt` defaults to the current UTC time on insert.

The `Email` column should be unique in practice (the `AuthController` checks for duplicates before inserting), but there's no `UNIQUE` constraint on the column itself yet. That's a gap I should fix.

Recipes

The biggest table. Most columns are straightforward, but two need explaining:

Ingredients is stored as a JSON string, not as a separate table. The JSON looks like `["200g pasta", "2 eggs", "100g bacon"]`. I went with JSON-in-a-column instead of a separate `Ingredients` table because the ingredients don't need to be queried independently. You never search "show me all recipes that use pasta" in this app (yet). If that feature gets added, a normalized `Ingredients` table with a foreign key back to `Recipes` would make more sense.

Instructions is also JSON. Each instruction is an object with a title and description: `[{"title": "Boil the pasta", "description": "Fill a pot with water..."}]`. Same reasoning as ingredients. The instructions are always read and written as a complete list alongside their recipe, never individually.

UserId links a recipe to the user who created it. It's nullable because some seeded recipes don't belong to any user. Once the auth system is fully wired to the frontend, new recipes will always have a `UserId` set.

ShoppingItems

Simple table for the shopping list feature. Each item has a name, quantity (stored as a string because users type things like "2 cans" or "500g"), a price in decimal, and an `IsChecked` boolean for the checkbox state. `UserId` will eventually filter shopping items per user, but right now the app shows all items to everyone since login isn't connected to the frontend yet.

Relationships

There are two relationships in the current schema:

- **Users to Recipes:** One-to-many. One user can create many recipes. The `UserId` foreign key on `Recipes` points back to `Users.Id`. Currently nullable since some recipes are seeded without an owner.
- **Users to ShoppingItems:** One-to-many. One user has many shopping items. The `UserId` foreign key on `ShoppingItems` points to `Users.Id`. Also nullable for the same reason.

There's no many-to-many relationship yet. If meal plans get a table, it'll likely be a join table between `Users` and `Recipes` with a day-of-week column, creating a many-to-many through a `MealPlan` table.

What I chose and why

JSON columns instead of normalized tables for ingredients/instructions. The trade-off is that I can't query individual ingredients with SQL. But for this app, ingredients and instructions are always read as a complete blob alongside their recipe. Normalizing them would mean extra JOIN queries on every recipe fetch and more repository code for something that doesn't need to be queried separately. If recipe search-by-ingredient becomes a feature, I'd reconsider this.

NVARCHAR for Quantity instead of a numeric type. Users type quantities like "a handful," "2 large," or "500g." Forcing a numeric type would mean splitting into amount + unit columns, which adds complexity for very little gain in a student meal planning app.

No foreign key constraints in the SQL yet. The relationships exist logically (`UserId` columns reference `Users.Id`), but I haven't added `FOREIGN KEY` constraints to the `CREATE TABLE` statements. That's something I should add. Without them, SQL Server won't stop me from inserting a recipe with a `UserId` that doesn't exist in the `Users` table.

[NOTE: If you've added any FK constraints since this was written, update this section.]

Tables I'll probably need later

These don't exist yet, but the app will likely need them:

- **MealPlans** (Id, UserId, RecipeId, DayOfWeek, MealType) for persisting the weekly meal planner. Right now meal plans only live in React state and disappear on page refresh.
- **SavedRecipes** (UserId, RecipeId) if I add a "save to favorites" feature. Simple join table.

SQL CREATE statements

For reference, here are the CREATE TABLE statements used to set up the database.

```
CREATE TABLE Users (  
  Id INT PRIMARY KEY IDENTITY(1,1),  
  Email NVARCHAR(255) NOT NULL,  
  PasswordHash NVARCHAR(255) NOT NULL,  
  UserName NVARCHAR(100) NOT NULL,  
  GoogleId NVARCHAR(255),  
  CreatedAt DATETIME DEFAULT GETUTCDATE()  
);
```

```
CREATE TABLE Recipes (  
  Id INT PRIMARY KEY IDENTITY(1,1),  
  Title NVARCHAR(255) NOT NULL,  
  Description NVARCHAR(MAX),  
  PrepTime INT,  
  CookTime INT,  
  Servings INT,  
  Tag NVARCHAR(100),  
  ImageUrl NVARCHAR(500),  
  EstimatedBudget DECIMAL(10,2),  
  Category NVARCHAR(100),  
  Ingredients NVARCHAR(MAX),  
  Instructions NVARCHAR(MAX),  
  UserId INT,  
  Tips NVARCHAR(MAX),  
  Calories INT,  
  Protein INT,  
  Carbs INT,  
  Fats INT,  
  CreatedAt DATETIME DEFAULT GETUTCDATE()  
);
```

```
CREATE TABLE ShoppingItems (  
  Id INT PRIMARY KEY IDENTITY(1,1),  
  Name NVARCHAR(255) NOT NULL,
```

```
Quantity NVARCHAR(100),  
Price DECIMAL(10,2),  
IsChecked BIT DEFAULT 0,  
UserId INT  
);
```

[NOTE: If you've changed these tables since setting them up, double-check the column types match what's actually in your database.]